9.4 Sorting with a Priority Queue

In defining the priority queue ADT, we noted that any type of object can be used as a key, but that any pair of keys must be comparable to each other, and that the set of keys be naturally ordered. In Python, it is common to rely on the $<$ operator to define such an order, in which case the following properties must be satisfied:

- **Irreflexive property**: $k \not< k$.
- **Transitive property**: if $k_1 < k_2$ and $k_2 < k_3$, then $k_1 < k_3$.

Formally, such a relationship defines what is known as a ***strict weak order***, as it allows for keys to be considered equal to each other, but the broader equivalence classes are ***totally ordered***, as they can be uniquely arranged from smallest to largest due to the transitive property.

As our first application of priority queues, we demonstrate how they can be used to sort a collection C of comparable elements. That is, we can produce a sequence of elements of C in increasing order (or at least in nondecreasing order if there are duplicates). The algorithm is quite simple—we insert all elements into an initially empty priority queue, and then we repeatedly call remove_min to retrieve the elements in nondecreasing order.

An implementation of this algorithm is given in Code Fragment 9.7, assuming that C is a positional list. (See Chapter 7.4.) We use an original element of the collection as both a key and value when calling P.add(element, element).

```
1  def pq_sort(C):
2    """Sort a collection of elements stored in a positional list."""
3    n = len(C)
4    P = PriorityQueue( )
5    for j in range(n):
6      element = C.delete(C.first( ))
7      P.add(element, element)          # use element as key and value
8    for j in range(n):
9      (k,v) = P.remove_min( )
10     C.add_last(v)                    # store smallest remaining element in C
```

Code Fragment 9.7: An implementation of the pq_sort function, assuming an appropriate implementation of a PriorityQueue class. Note that each element of the input list C serves as its own key in the priority queue P.

With a minor modification to this code, we can provide more general support, sorting elements according to an ordering other than the default. For example, when working with strings, the $<$ operator defines a ***lexicographic ordering***, which is an extension of the alphabetic ordering to Unicode. For example, we have that `'12'` $<$ `'4'` because of the order of the first character of each string, just as

'apple' < 'banana'. Suppose that we have an application in which we have a list of strings that are all known to represent integral values (e.g., '12'), and our goal is to sort the strings according to those integral values.

In Python, the standard approach for customizing the order for a sorting algorithm is to provide, as an optional parameter to the sorting function, an object that is itself a one-parameter function that computes a key for a given element. (See Sections 1.5 and 1.10 for a discussion of this approach in the context of the built-in max function.) For example, with a list of (numeric) strings, we might wish to use the value of int(s) as a key for a string s of the list. In this case, the constructor for the int class can serve as the one-parameter function for computing a key. In that way, the string '4' will be ordered before string '12' because its key int('4') < int('12'). We leave it as an exercise to support such an optional key parameter for the pq_sort function. (See Exercise C-9.46.)

9.4.1 Selection-Sort and Insertion-Sort

Our pq_sort function works correctly given any valid implementation of the priority queue class. However, the running time of the sorting algorithm depends on the running times of the operations add and remove_min for the given priority queue class. We next discuss a choice of priority queue implementations that in effect cause the pq_sort computation to behave as one of several classic sorting algorithms.

Selection-Sort

If we implement P with an unsorted list, then Phase 1 of pq_sort takes $O(n)$ time, for we can add each element in $O(1)$ time. In Phase 2, the running time of each remove_min operation is proportional to the size of P. Thus, the bottleneck computation is the repeated "selection" of the minimum element in Phase 2. For this reason, this algorithm is better known as ***selection-sort***. (See Figure 9.7.)

As noted above, the bottleneck is in Phase 2 where we repeatedly remove an entry with smallest key from the priority queue P. The size of P starts at n and incrementally decreases with each remove_min until it becomes 0. Thus, the first operation takes time $O(n)$, the second one takes time $O(n-1)$, and so on. Therefore, the total time needed for the second phase is

$$O(n+(n-1)+\cdots+2+1) = O\left(\textstyle\sum_{i=1}^{n} i\right).$$

By Proposition 3.3, we have $\sum_{i=1}^{n} i = n(n+1)/2$. Thus, Phase 2 takes time $O(n^2)$, as does the entire selection-sort algorithm.

		Collection C	*Priority Queue P*
Input		$(7,4,8,2,5,3)$	$()$
Phase 1	(a)	$(4,8,2,5,3)$	(7)
	(b)	$(8,2,5,3)$	$(7,4)$
	$\vdots$	$\vdots$	$\vdots$
	(f)	$()$	$(7,4,8,2,5,3)$
Phase 2	(a)	(2)	$(7,4,8,5,3)$
	(b)	$(2,3)$	$(7,4,8,5)$
	(c)	$(2,3,4)$	$(7,8,5)$
	(d)	$(2,3,4,5)$	$(7,8)$
	(e)	$(2,3,4,5,7)$	(8)
	(f)	$(2,3,4,5,7,8)$	$()$

Figure 9.7: Execution of selection-sort on collection $C = (7,4,8,2,5,3)$.

Insertion-Sort

If we implement the priority queue P using a sorted list, then we improve the running time of Phase 2 to $O(n)$, for each remove_min operation on P now takes $O(1)$ time. Unfortunately, Phase 1 becomes the bottleneck for the running time, since, in the worst case, each add operation takes time proportional to the current size of P. This sorting algorithm is better known as ***insertion-sort*** (see Figure 9.8); in fact, our implementation for adding an element to a priority queue is almost identical to a step of insertion-sort as presented in Section 7.5.

The worst-case running time of Phase 1 of insertion-sort is

$$O(1+2+\ldots+(n-1)+n) = O\left(\textstyle\sum_{i=1}^{n} i\right).$$

Again, by Proposition 3.3, this implies a worst-case $O(n^2)$ time for Phase 1, and thus, the entire insertion-sort algorithm. However, unlike selection-sort, insertion-sort has a *best-case* running time of $O(n)$.

		Collection C	*Priority Queue P*
Input		$(7,4,8,2,5,3)$	$()$
Phase 1	(a)	$(4,8,2,5,3)$	(7)
	(b)	$(8,2,5,3)$	$(4,7)$
	(c)	$(2,5,3)$	$(4,7,8)$
	(d)	$(5,3)$	$(2,4,7,8)$
	(e)	(3)	$(2,4,5,7,8)$
	(f)	$()$	$(2,3,4,5,7,8)$
Phase 2	(a)	(2)	$(3,4,5,7,8)$
	(b)	$(2,3)$	$(4,5,7,8)$
	$\vdots$	$\vdots$	$\vdots$
	(f)	$(2,3,4,5,7,8)$	$()$

Figure 9.8: Execution of insertion-sort on collection $C = (7,4,8,2,5,3)$.

9.4.2 Heap-Sort

As we have previously observed, realizing a priority queue with a heap has the advantage that all the methods in the priority queue ADT run in logarithmic time or better. Hence, this realization is suitable for applications where fast running times are sought for all the priority queue methods. Therefore, let us again consider the pq_sort scheme, this time using a heap-based implementation of the priority queue.

During Phase 1, the ith add operation takes $O(\log i)$ time, since the heap has i entries after the operation is performed. Therefore this phase takes $O(n \log n)$ time. (It could be improved to $O(n)$ with the bottom-up heap construction described in Section 9.3.6.)

During the second phase of pq_sort, the jth remove_min operation runs in $O(\log(n-j+1))$, since the heap has $n-j+1$ entries at the time the operation is performed. Summing over all j, this phase takes $O(n \log n)$ time, so the entire priority-queue sorting algorithm runs in $O(n \log n)$ time when we use a heap to implement the priority queue. This sorting algorithm is better known as ***heap-sort***, and its performance is summarized in the following proposition.

Proposition 9.4: *The heap-sort algorithm sorts a collection C of n elements in $O(n \log n)$ time, assuming two elements of C can be compared in $O(1)$ time.*

Let us stress that the $O(n \log n)$ running time of heap-sort is considerably better than the $O(n^2)$ running time of selection-sort and insertion-sort (Section 9.4.1).

Implementing Heap-Sort In-Place

If the collection C to be sorted is implemented by means of an array-based sequence, most notably as a Python list, we can speed up heap-sort and reduce its space requirement by a constant factor using a portion of the list itself to store the heap, thus avoiding the use of an auxiliary heap data structure. This is accomplished by modifying the algorithm as follows:

1. We redefine the heap operations to be a *maximum-oriented* heap, with each position's key being at least as *large* as its children. This can be done by recoding the algorithm, or by adjusting the notion of keys to be negatively oriented. At any time during the execution of the algorithm, we use the left portion of C, up to a certain index $i-1$, to store the entries of the heap, and the right portion of C, from index i to $n-1$, to store the elements of the sequence. Thus, the first i elements of C (at indices $0, \ldots, i-1$) provide the array-list representation of the heap.
2. In the first phase of the algorithm, we start with an empty heap and move the boundary between the heap and the sequence from left to right, one step at a time. In step i, for $i = 1, \ldots, n$, we expand the heap by adding the element at index $i-1$.

3. In the second phase of the algorithm, we start with an empty sequence and move the boundary between the heap and the sequence from right to left, one step at a time. At step i, for $i = 1, \ldots, n$, we remove a maximum element from the heap and store it at index $n - i$.

In general, we say that a sorting algorithm is ***in-place*** if it uses only a small amount of memory in addition to the sequence storing the objects to be sorted. The variation of heap-sort above qualifies as in-place; instead of transferring elements out of the sequence and then back in, we simply rearrange them. We illustrate the second phase of in-place heap-sort in Figure 9.9.

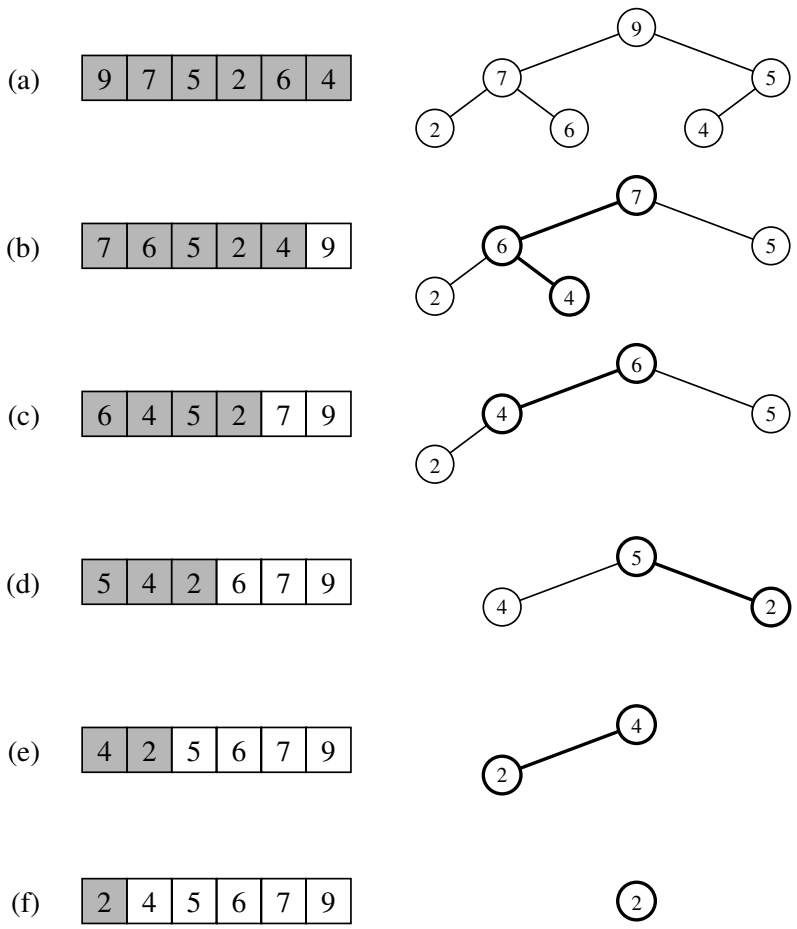


Figure 9.9: Phase 2 of an in-place heap-sort. The heap portion of each sequence representation is highlighted. The binary tree that each sequence (implicitly) represents is diagrammed with the most recent path of down-heap bubbling highlighted.